

2026 €€€ 超专业级别硬件能力认证第一轮
(C5P-S1) 提高级 C++语言试题

认证时间：2026 年 9 月 18 日 14:30~16:30

考生注意事项：

- 试题纸共有 15 页，答题纸共有 1 页，满分 100 分。请在答题纸上作答，写在试题纸上的一律无效。
- 不得使用任何电子设备（如计算器、手机、电子词典等）或查阅任何书籍资料。

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 下列 Linux 命令中，用于连接并显示文件内容的是（ ）
A. cp
B. cat
C. ping
D. touch
2. 已知某用二叉链表存储的哈夫曼树共有 m 个叶子结点，则该哈夫曼树中总共有多少个空指针域（ ）
A. m
B. $m+1$
C. $2m$
D. $2m+1$
3. 对于一棵包含 n 个结点的完全二叉树，对每个叶子结点向上访问直到根结点，统计路径上经过的结点数之和。总时间复杂度为（ ）
A. $O(n)$
B. $O(n \log n)$
C. $O(n^2)$
D. $O(n^2 \log n)$
4. 小 h 从原点 $(0,0)$ 出发，每一步向右或向上走，到达点 $(5,5)$ ，且始终满足 $x \geq y$ 。满足条件的路径有多少条（ ）
A. 252
B. 90
C. 132
D. 42
5. 用两个栈模拟一个队列，入队时元素统一压入栈 A。正确的出队操作是（ ）
A. B 为空时将 A 全部转移到 B，再从 B 弹出；B 非空时直接从 B 弹出
B. 无论 B 是否为空，都先将 A 全部转移到 B，再从 B 弹出

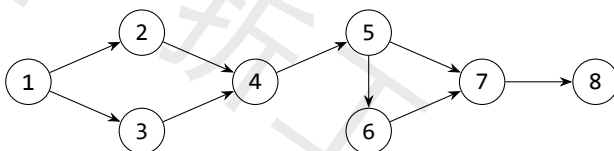
- C. 直接删除 A 的栈底元素
D. 直接从 A 弹出栈顶元素
6. 从集合 $\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ 中选出 4 个不同的数, 要求任意两个被选数都不相邻。不同的选法共有 ()
A. 15
B. 20
C. 35
D. 70
7. 在 C++ 中, 关于 `std::set<int>` 与 `std::map<int, int>` 的说法, 正确的是 ()
A. `set` 中的元素可以重复
B. `map` 按插入顺序存储键值对
C. `set` 自动有序, 插入、删除、查找复杂度均为 $O(\log n)$
D. `map` 的键可以重复
8. 关于有向无环图的拓扑排序, 下列说法正确的是 ()
A. 任何有向无环图的拓扑排序都唯一
B. 含有环的有向图也可以进行拓扑排序
C. 若有边 $u \rightarrow v$, 则任一拓扑序中 u 在 v 之前
D. 拓扑排序就是深度优先遍历序列
9. KMP 算法中, `next` 数组的主要作用是 ()
A. 记录模式串失配后应回溯到的位置
B. 记录主串每个字符出现的次数
C. 记录主串的后缀数组
D. 与模式串无关地跳过主串位置
10. 对一个 n 个顶点、 m 条边的带权有向图使用不带堆优化的 Dijkstra 算法, 时间复杂度为 ()
A. $O(n^2)$
B. $O(m \log n)$
C. $O((m+n) \log n)$
D. $O(mn)$
11. 已知 20260913 是质数, 表达式 $(1/2 + 1/3 + \dots + 1/20260912)^{20260913} \bmod 20260913$ 的值为 ()
A. 0
B. 1
C. $2^{20260913} \bmod 20260913$
D. 20260912
12. 设 $p = 20260913$ 为质数。在模 p 意义下,

$$\sum_{i=1}^{p-2} \frac{1}{i(i+1)}$$

等于 ()

- A. 0
- B. 1
- C. 2
- D. $p - 2$

13. 对下图恰好删去两条不同的边, 使得仍存在从 1 到 8 的有向路径。删边方案共有 () 种



- A. 12
- B. 15
- C. 18
- D. 21

14. 1 到 200 之间, 不能被 2、3、7 中任何一个整除的整数共有 () 个

- A. 57
- B. 58
- C. 59
- D. 60

15. 已知如下 C++14 代码定义 Ackermann 函数:

```
01 long long A(int m, long long n) {  
02     if (m == 0) return n + 1;  
03     if (n == 0) return A(m - 1, 1);  
04     return A(m - 1, A(m, n - 1));  
05 }
```

则二元组 $(A(3, 3), A(3, 10^{18}) \bmod 101)$ 的值为 ()

- A. (61, 5)
- B. (61, 8)
- C. (31, 5)
- D. (31, 8)

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填 $\checkmark$ ，错误填 $\times$ ；除特殊说明外，判断题 1.5 分，选择题 3 分，共计 40 分）

(1)

```
01#include <bits/stdc++.h>
02using namespace std;
03const int N = 2e5 + 5;
04int n, pi[N], occ[N], dep[N];
05string s;
06
07int main() {
08    cin >> s;
09    n = s.size();
10    for (int i = 1; i < n; ++i) {
11        int j = pi[i - 1];
12        while (j && s[i] != s[j]) j = pi[j - 1];
13        if (s[i] == s[j]) ++j;
14        pi[i] = j;
15    }
16    for (int i = 0; i < n; ++i) ++occ[pi[i]];
17    for (int i = n - 1; i > 0; --i)
18        occ[pi[i - 1]] += occ[i];
19    for (int i = 0; i <= n; ++i) ++occ[i];
20    long long A = 0, B = 0;
21    for (int i = 1; i <= n; ++i) {
22        dep[i] = dep[pi[i - 1]] + 1;
23        A += occ[i];
24        B += dep[i];
25    }
26    cout << occ[n / 2] << ' ' << dep[n] << ' ' << A
27    << '\n';
28}
```

• 判断题

16. 对 $0 \leq i < n$, $pi[i]$ 等于 $s[0..i]$ 的最长真前缀与后缀的公共长度。()
A. 正确
B. 错误
17. 执行求解 pi 的循环 $\text{for (int } i = 1; i < n; ++i)$ 结束后, $pi[0]$ 到 $pi[n-1]$ 已

经全部求出。()

- A. 正确
- B. 错误

18. 输入字符串为 ababa。执行统计 occ 的三段循环后, occ[3] 等于 2。()

- A. 正确
- B. 错误

• 单选题

19. 输入 aaaaaa 时, 程序输出为 ()

- A. 3 6 21
- B. 4 5 21
- C. 4 6 21
- D. 4 6 15

20. (4 分) 输入 abacababacabab 时, 程序输出为 ()

- A. 1 3 30
- B. 2 3 32
- C. 2 4 32
- D. 3 3 34

(2)

```
01#include <bits/stdc++.h>
02using namespace std;
03const int P = 998244353, M = 10;
04int n, m, ban[105], f[2][1 << M];
05vector<int> st;
06
07int main() {
08    cin >> n >> m;
09    for (int i = 1; i <= n; ++i) {
10        string s; cin >> s;
11        for (int j = 0; j < m; ++j)
12            if (s[j] == '#') ban[i] |= 1 << j;
13    }
14    for (int s = 0; s < (1 << m); ++s)
15        if ((s & (s << 1)) == 0) st.emplace_back(s);
16    f[0][0] = 1;
17    for (int i = 1; i <= n; ++i) {
18        int pre = i & 1, cur = pre ^ 1;
```

```

19     int mask = pre ^ cur;
20     pre ^= mask;
21     cur ^= mask;
22     memset(f[cur], 0, sizeof(f[cur]));
23     for (auto&& s : st) {
24         if (s & ban[i]) continue;
25         for (auto&& t : st) {
26             if (s & t) continue;
27             f[cur][s] += f[pre][t];
28             if (f[cur][s] >= P) f[cur][s] -= P;
29         }
30     }
31 }
32 int ans = 0;
33 for (auto&& s : st) {
34     ans += f[n & 1][s];
35     if (ans >= P) ans -= P;
36 }
37 cout << ans << '\n';
38 }

```

• 判断题

21. 程序输出从所有可选位置中选取若干位置，使任意两个被选位置不共边的方案数对 P 取模后的结果。()

- A. 正确
- B. 错误

22. 处理第 i 行时，令 s 表示第 i 行的选取状态、 t 表示第 $i-1$ 行的选取状态。若第 i 行第 j 列与第 $i-1$ 行第 $j+1$ 列同时被选中，程序不会因这两个对角线相邻位置而跳过该状态转移。()

- A. 正确
- B. 错误

23. (2 分) 处理第 i 行时，执行 $\text{pre} = i \& 1$ 、 $\text{cur} = \text{pre} \text{ XOR } 1$ 及后面的两次下标交换后， pre 指向第 $i-1$ 行结果所在的滚动数组， cur 指向第 i 行要写入的滚动数组。()

- A. 正确
- B. 错误

• 单选题

24. 记 $V = \text{st.size}()$ ，则程序的时间复杂度为 ()

- A. $O(nV)$

B. $O(nV^2 + 2^m)$

C. $O(n2^{2m})$

D. $O(n^2V)$

25. 输入为 $n=2, m=3$, 两行均为 ... 时, 程序输出为 ()

A. 15

B. 16

C. 17

D. 18

26. 在处理第 i 行之前, 若 $s \in st$, 则 $f[pre][s]$ 表示 ()

A. 前 i 行中最后一行状态为 s 的合法方案数

B. 前 $i-1$ 行中最后一行状态为 s 的合法方案数

C. 长度为 m 的合法行状态总数

D. 第 i 行所有可选位置的数量

(3)

```
01#include <bits/stdc++.h>
02using namespace std;
03using ll = long long;
04const ll P = 1000000007;
05const int N = 2e5 + 5;
06int n, q;
07ll a[N];
08struct Node { ll sum, mul, add; } tr[N << 2];
09
10void build(int p, int l, int r) {
11    tr[p].mul = 1;
12    if (l == r) { tr[p].sum = a[l] % P; return; }
13    int m = (l + r) >> 1;
14    build(p << 1, l, m); build(p << 1 | 1, m + 1, r)
15    ;
16    tr[p].sum = (tr[p << 1].sum + tr[p << 1 | 1].sum
17    ) % P;
18}
19void apply(int p, int l, int r, ll k, ll b) {
20    tr[p].sum = (tr[p].sum * k + (r - l + 1) * b) %
21    P;
22    tr[p].mul = tr[p].mul * k % P;
```

```

20     tr[p].add = (tr[p].add * k + b) % P;
21 }
22 void push(int p, int l, int r) {
23     if (l == r) return;
24     int m = (l + r) >> 1;
25     apply(p << 1, l, m, tr[p].mul, tr[p].add);
26     apply(p << 1 | 1, m + 1, r, tr[p].mul, tr[p].add
27 );
28     std::exchange(tr[p].mul, 1);
29     std::exchange(tr[p].add, 0);
30 }
31 void upd(int p, int l, int r, int ql, int qr, ll k,
32         ll b) {
33     if (ql <= l && r <= qr) { apply(p, l, r, k, b);
34         return; }
35     push(p, l, r);
36     int m = (l + r) >> 1;
37     if (ql <= m) upd(p << 1, l, m, ql, qr, k, b);
38     if (qr > m) upd(p << 1 | 1, m + 1, r, ql, qr, k,
39         b);
40     tr[p].sum = (tr[p << 1].sum + tr[p << 1 | 1].sum
41 ) % P;
42 }
43 ll qry(int p, int l, int r, int ql, int qr) {
44     if (ql <= l && r <= qr) return tr[p].sum;
45     push(p, l, r);
46     int m = (l + r) >> 1;
47     ll s = 0;
48     if (ql <= m) s = (s + qry(p << 1, l, m, ql, qr))
49         % P;
50     if (qr > m) s = (s + qry(p << 1 | 1, m + 1, r,
51         ql, qr)) % P;
52     return s;
53 }
54 int main() {

```



```

48     cin >> n >> q;
49     for (int i = 1; i <= n; ++i) cin >> a[i];
50     build(1, 1, n);
51     while (q--) {
52         int op, l, r; cin >> op >> l >> r;
53         if (!(op ^ 1)) {
54             ll k, b; cin >> k >> b;
55             upd(1, 1, n, l, r, k, b);
56         } else cout << qry(1, 1, n, l, r) << '\n';
57     }
58 }

```

• 判断题

27. 同一线段树结点 p 先后执行 $\text{apply}(p, l, r, 2, 1)$ 、 $\text{apply}(p, l, r, 3, 4)$ 后, 其懒标记表示的变换为 $x \mapsto 6x + 7$ 。()

- A. 正确
- B. 错误

28. 假设主函数代码第 55 行依次执行两条更新: 对区间 $[2, 4]$ 先执行 $x \leftarrow 2x + 1$, 再执行 $x \leftarrow x + 3$ 。若只交换这两次 $\text{upd}(\dots)$ 调用的先后顺序, 最终数组一定不变。()

- A. 正确
- B. 错误

29. 程序的时间复杂度为 $O((n + q) \log n)$ 。()

- A. 正确
- B. 错误

• 单选题

30. 当 upd 函数执行到代码第 32 行, 且当前结点区间 $[l, r]$ 未被更新区间 $[ql, qr]$ 完全包含时, $\text{push}(p, l, r)$ 的作用是 ()

- A. 将当前结点的懒标记表示的更新下传给两个子结点, 再递归处理可能相交的子区间
- B. 将两个子结点的区间和直接赋给当前结点, 并清空子结点
- C. 将更新区间扩大为当前结点区间, 使当前结点可以直接调用 apply
- D. 只在查询操作中统计当前结点的区间和, 不改变任何懒标记

31. 当 qry 函数执行到代码第 39 行且 $ql \leq l \ \&\& \ r \leq qr$ 成立时, 程序直接返回 $\text{tr}[p].\text{sum}$, 不执行第 40 行的 $\text{push}(p, l, r)$ 。这样做的原因是 ()

- A. 当前结点的区间和已经包含其待下传更新, 且查询不需要访问该结点的子结点
- B. 查询操作不需要考虑任何懒标记, 子结点中的值一定比当前结点准确
- C. 只有叶结点才允许返回 $\text{tr}[p].\text{sum}$, 所以此处必然不会继续递归
- D. push 会改变查询区间, 因此完全包含时必须跳过所有区间更新

32. (4 分) 初始数组为 2 1 3 5 4 6, 依次执行 1 2 6 2 1、1 1 4 3 2、1 3 5 1 4、2 2

6, 最后一次操作输出为 ()

- A. 97
- B. 101
- C. 103
- D. 107

三、完善程序（单选题，每小题 3 分，共计 30 分）

(1)（三色序列）给定一个长度为 n 、只含字符 R,G,Y 的字符串。每次可以交换两个相邻字符。求使相邻字符均不同所需的最少交换次数；若无法做到，输出 -1。 $1 \leq n \leq 400$ 。

```
01#include <bits/stdc++.h>
02using namespace std;
03const int N = 405, INF = 1e9;
04int n, cnt[3], pos[3][N], pre[3][N];
05int f[2][N][N][4];
06string s;
07int main() {
08    cin >> n >> s;
09    for (int i = 1; i <= n; ++i) {
10        int c;
11        if (s[i-1] == 'R') c = 0;
12        else if (s[i-1] == 'G') c = 1;
13        else c = 2;
14        for (int t = 0; t < 3; ++t)
15            pre[t][i] = ①;
16        pos[c][++cnt[c]] = i;
17        ++pre[c][i];
18    }
19    memset(f, 0x3f, sizeof(f));
20    f[0][0][0][3] = ②;
21    for (int len = 0; len < n; ++len) {
22        int cur = len & 1, nxt = cur ^ 1;
23        memset(f[nxt], 0x3f, sizeof(f[nxt]));
24        for (int r = 0; r <= cnt[0]; ++r)
25            for (int g = 0; g <= cnt[1]; ++g) {
26                int y = ③;
27                if (y < 0 || y > cnt[2]) continue;
28                int used[3] = {r, g, y};
29                for (int last = 0; last < 4; ++last) {
30                    int val = f[cur][r][g][last];
31                    if (val >= INF) continue;
32                    for (int c = 0; c < 3; ++c) {
```

```

33         if (____④____) continue;
34         int x = pos[c][used[c]+1];
35         int w = x-len-1;
36         for (int t = 0; t < 3; ++t)
37             w += max(0, used[t]-pre[t][x
38     ]);
39
40         int nr = r, ng = g;
41         if (c == 0) ++nr;
42         if (c == 1) ++ng;
43         int nv = ____⑤____;
44         f[nxt][nr][ng][c] =
45             min(f[nxt][nr][ng][c], nv);
46     }
47 }
48 int ans = INF, z = n & 1;
49 for (int c = 0; c < 3; ++c) {
50     int v = f[z][cnt[0]][cnt[1]][c];
51     ans = min(ans, v);
52 }
53 cout << (ans >= INF ? -1 : ans) << '\n';
54 }

```

33. ①处应填 ()
 A. pre[t][i] B. pre[t][i-1]
 C. pre[t][i+1] D. pre[i][t]
34. ②处应填 ()
 A. INF B. -1
 C. 0 D. 1
35. ③处应填 ()
 A. len-r-g B. r+g-len
 C. n-r-g D. len-r+g
36. ④处应填 ()
 A. c!=last || used[c]==cnt[c] B. c==last && used[c]==cnt[c]
 C. c==last || used[c]==cnt[c] D. c!=last && used[c]<cnt[c]

37. ⑤处应填 ()

- A. val B. w
C. val+w D. val+len-w

(2) (盒子) 有 n 个体积互不相同的盒子和 k 个物品。每个盒子中恰放一个物品或一个体积更小的盒子；把体积为 x 的盒子放入体积为 y 的盒子需要 $y - x$ 单位缓冲材料。所有盒子均须使用。先求所需缓冲材料的最小总量；随后有 q 次删除，每次永久删除一个指定盒子并再次求答案。保证始终至少剩下 k 个盒子。 $1 \leq n \leq 2 \cdot 10^5$ 。

```
01#include <bits/stdc++.h>
02using namespace std;
03using ll = long long;
04const int N = 2e5 + 5;
05int n, k, q, need;
06ll c[N], ans;
07set<ll> box;
08multiset<ll> lo, hi;
09
10void fix() {
11    while ((int)lo.size() > need) {
12        auto it = ②;
13        ans -= *it; hi.insert(*it); lo.erase(it);
14    }
15    while ((int)lo.size() < need) {
16        auto it = hi.begin();
17        ans += *it; lo.insert(*it); hi.erase(it);
18    }
19    while (!lo.empty() && !hi.empty()) {
20        auto x = prev(lo.end()), y = hi.begin();
21        if (③) break;
22        ll vx = *x, vy = *y;
23        ans += ④;
24        lo.erase(x); hi.erase(y);
25        lo.insert(vy); hi.insert(vx);
26    }
27}
28void add_gap(ll x) { lo.insert(x); ans += x; }
29void del_gap(ll x) {
```

```

30     auto it = lo.find(x);
31     if (it != lo.end()) { ans -= x; lo.erase(it); }
32     else hi.erase(hi.find(x));
33 }
34 int main() {
35     cin >> n >> k;
36     for (int i = 1; i <= n; ++i) {
37         cin >> c[i];
38         box.insert(c[i]);
39     }
40     if (box.size() > 1) {
41         auto x = box.begin(), y = next(x);
42         for (; y != box.end(); ++x, ++y)
43             add_gap(*y - *x);
44     }
45     need = ①;
46     fix(); cout << ans << '\n';
47     cin >> q;
48     while (q--) {
49         int d; cin >> d;
50         ll v = c[d], l = 0, r = 0;
51         auto it = box.find(v);
52         auto rt = next(it);
53         bool hl = it != box.begin();
54         bool hr = rt != box.end();
55         if(hl){ l=*prev(it); del_gap(v-l); }
56         if(hr){ r=*rt; del_gap(r-v); }
57         box.erase(it);
58         --need;
59         if (hl && hr) add_gap(⑤);
60         fix(); cout << ans << '\n';
61     }
62 }

```

38. ①处应填 ()

- A. $n-k$ B. $k-1$

C. $n-k+1$ D. $n-1$

39. ②处应填 ()

A. `lo.begin()` B. `prev(lo.end())`

C. `hi.begin()` D. `prev(hi.end())`

40. ③处应填 ()

A. `*x<*y` B. `*x<=*y`

C. `*x>*y` D. `*x>=*y`

41. ④处应填 ()

A. `vx-vy` B. `vy-vx`

C. `vx+vy` D. `-vx`

42. ⑤处应填 ()

A. `v-l` B. `r-v`

C. `r-l` D. `r+l-v`